

Back log

Cliente - Criação do ApiCallService:

- Prevenção de Socket Exhaustion: O uso de `using(var httpClient = new HttpClient())` pode esgotar as portas do servidor; o serviço centralizado usa o `IHttpClientFactory` de forma eficiente.
- Centralização da Base URL: Permite alterar o endereço da API em um único arquivo de configuração (`appsettings.json`) em vez de editar strings em cada controller.
- Eliminação de Código Duplicado (DRY) nos Controllers: Remove a necessidade de repetir a lógica de captura de token, criação de headers e verificação de status em todos os métodos.
- Tratamento de Erros Padronizado: Garante que falhas de rede ou de autorização sejam tratadas da mesma forma em toda a aplicação, facilitando o Debug.
- Gestão Automática de Autenticação: O serviço injeta o `Bearer Token` automaticamente antes de cada chamada, reduzindo o risco de esquecimento e erros manuais.
- Facilidade de Manutenção: Qualquer mudança na lógica de segurança ou serialização JSON é feita em um único ponto, refletindo-se instantaneamente em todo o projeto.
- Melhoria na Legibilidade: Os métodos dos Controllers ficam focados na lógica de negócio e navegação, tornando o código muito mais limpo e profissional.
- Testabilidade: Facilita a criação de testes unitários, permitindo simular (mockar) as respostas da API sem precisar de uma conexão real de rede.

API - Limpeza Controller (Criação da camada de Serviços)

API - Melhora de performance na gestão do histórico (centralização em uma tabela específica para o histórico)

API - Refatorar Delete e configurar Cascade no DbContext